Backend README: User CRUD API with PostgreSQL in Docker

A simple REST API for managing users, using Express, Prisma 7, and PostgreSQL running in Docker. Built for learning. This is the PostgreSQL version of the earlier SQLite backend. The frontend does not change.

Tech Stack

- **Node.js**: runs JavaScript on the server.
- **Express**: web framework that defines the API routes.
- **TypeScript**: JavaScript with types, run in development by tsx.
- **Prisma 7**: ORM. You describe tables in a schema file and call typed functions like findMany and create instead of writing SQL by hand.
- **PostgreSQL 16**: a real database server, running in a Docker container.
- **Driver adapter (@prisma/adapter-pg) and pg**: Prisma 7 needs an adapter, a bridge between Prisma and the database. The adapter uses the pg driver underneath.
- **cors**: lets the frontend (port 3000) call the backend (port 3001).
- **dotenv**: loads settings from the .env file.
- **Docker Desktop**: runs the PostgreSQL container on your computer.

SQLite vs PostgreSQL Comparison

	SQLite version	PostgreSQL version
Flow	Node.js -> Prisma -> adapter -> SQLite file	Node.js -> Prisma -> adapter -> pg -> PostgreSQL database
Adapter package	@prisma/adapter-better-sqlite3	@prisma/adapter-pg and pg
Where data lives	One file (dev.db)	A database server in a Docker container
Extra software needed	None	Docker Desktop
DATABASE_URL	file:./dev.db	postgresql://postgres:postgres@localhost:5432/usersdb
Prisma provider	sqlite	postgresql

	SQLite version	PostgreSQL version
Adapter code	new PrismaBetterSqlite3({ url })	new PrismaPg({ connectionString })
Hosting	Needs a persistent disk	Works with managed hosts such as Neon, Supabase, AWS RDS

Everything else is the same: the User model, the Express routes, the Postman tests, and the whole frontend.

Step 1: Install Docker Desktop (Windows)

1. Go to docker.com, open Products, choose **Docker Desktop**, and download the Windows version. Choose **AMD64** for Intel and AMD computers. ARM64 is only for Snapdragon-type chips.
2. Run Docker Desktop Installer. Tick **WSL 2**. Leave Hyper-V, Docker VMM, and Windows containers unticked.
3. If the installer says the Virtual Machine Platform feature is missing: open "Turn Windows features on or off", tick **Virtual Machine Platform** and **Windows Subsystem for Linux**, click OK, and restart.
4. If the error continues, open Terminal as administrator and run `wsl --install`, then restart again.
5. Open Docker Desktop yourself after the restart and wait until the engine shows as running (green icon at the bottom left). The WSL welcome screen can be closed.

Step 2: Start PostgreSQL in Docker

Run this on one line in the terminal:

```
docker run --name usersdb -e POSTGRES_USER=postgres -e
POSTGRES_PASSWORD=postgres -e POSTGRES_DB=usersdb -p 5432:5432 -d postgres:16
```

What each part means:

- **docker run**: create and start a new container.
- **--name usersdb**: the container name, easy to find in Docker Desktop.

- **-e POSTGRES_USER, POSTGRES_PASSWORD, POSTGRES_DB**: environment variables PostgreSQL reads on first start to create the username, password, and database name.
- **-p 5432:5432**: connects a port on your computer (left) to the port inside the container (right). Change only the left number if 5432 is already used, for example 5433:5432. The right number stays 5432.
- **-d**: detached, runs in the background.
- **postgres:16**: the image, PostgreSQL version 16.

Check it with `docker ps`. You should see usersdb in the list.

Step 3: Create the Project

```
mkdir backend_postgresql
cd backend_postgresql
npm init -y
```

Step 4: Install Packages

Runtime packages:

```
npm install express cors dotenv @prisma/client @prisma/adapter-pg pg
```

Dev packages (the -D flag means needed only while building):

```
npm install -D typescript tsx prisma @types/node @types/express @types/cors
@types/pg
```

- **Prisma client**: code generated from your schema, giving typed functions like findMany and create.
- **adapter-pg**: the bridge between Prisma and PostgreSQL.
- **pg**: the PostgreSQL driver the adapter uses.
- **tsx**: runs TypeScript files directly.
- **@types packages**: type information for Node, Express, cors, and pg.

Step 5: Configure TypeScript

```
npx tsc --init
```

In tsconfig.json set:

- "module": "nodenext"
- "target": "esnext"
- "moduleResolution": "nodenext"
- "types": ["node"]

Meaning: esnext keeps modern JavaScript features. nodenext follows modern Node.js module rules. types node fixes the "cannot find name process" error.

In **package.json** (a different file) add this line near the top:

```
"type": "module",
```

This tells Node the project uses modern import syntax.

Step 6: Initialize Prisma for PostgreSQL

```
npx prisma init --datasource-provider postgresql
```

This creates the prisma folder with schema.prisma, a Prisma config file, and a .env file. The config file needs no edits. It reads DATABASE_URL from .env and is used only by Prisma commands.

Step 7: Environment Variables (.env)

```
DATABASE_URL="postgresql://postgres:postgres@localhost:5432/usersdb"
PORT=3001
```

The URL pattern is: postgresql://USER:PASSWORD@HOST:PORT/DATABASE. The values come from the docker run command. If you changed the left port number, use it here.

Step 8: Prisma Schema

Keep the generator and datasource blocks (provider should be postgresql) and add this at the bottom of prisma/schema.prisma:

```
model User {
  id        Int      @id @default(autoincrement())
  name      String
  email     String   @unique
  createdAt DateTime @default(now())
}
```

- model User becomes the User table.
- @id @default(autoincrement()) makes the primary key count 1, 2, 3 automatically.
- @unique blocks duplicate emails.
- @default(now()) fills in the creation time.

Step 9: Create the Table

```
npx prisma migrate dev --name init
npx prisma generate
```

- **migrate dev** reads the schema, writes the SQL to create the User table, saves it in prisma/migrations, and runs it on PostgreSQL. **--name init** is just a label for this first migration.
- **generate** creates the typed Prisma client in the generated folder. Run it again after every schema change.

Step 10: The Server (src/index.ts)

Create a src folder next to package.json and prisma, and put index.ts inside it.

```
import "dotenv/config";
import express from "express";
import cors from "cors";
import { PrismaPg } from "@prisma/adapter-pg";
import { PrismaClient } from "../generated/prisma/client.js";

const adapter = new PrismaPg({ connectionString: process.env.DATABASE_URL!
```

```
});
const prisma = new PrismaClient({ adapter });

const app = express();
app.use(cors());
app.use(express.json());

app.get("/users", async (_req, res) => {
  const users = await prisma.user.findMany();
  res.json(users);
});

app.get("/users/:id", async (req, res) => {
  const user = await prisma.user.findUnique({
    where: { id: Number(req.params.id) },
  });
  if (!user) return res.status(404).json({ error: "User not found" });
  res.json(user);
});

app.post("/users", async (req, res) => {
  const { name, email } = req.body;
  if (!name || !email) {
    return res.status(400).json({ error: "Name and email are required" });
  }
  try {
    const user = await prisma.user.create({ data: { name, email } });
    res.status(201).json(user);
  } catch {
    res.status(409).json({ error: "Email already exists" });
  }
});

app.put("/users/:id", async (req, res) => {
  const { name, email } = req.body;
  try {
    const user = await prisma.user.update({
      where: { id: Number(req.params.id) },
      data: { name, email },
    });
    res.json(user);
  } catch {
    res.status(404).json({ error: "User not found or email already exists"
```

```
});
  }
});

app.delete("/users/:id", async (req, res) => {
  try {
    await prisma.user.delete({ where: { id: Number(req.params.id) } });
    res.status(204).send();
  } catch {
    res.status(404).json({ error: "User not found" });
  }
});

const PORT = process.env.PORT || 3001;
app.listen(PORT, () => console.log(`Server running on
http://localhost:${PORT}`));
```

Keep app.listen last, so the server starts after all routes are defined.

Step 11: Run It

In package.json, inside "scripts", add:

```
"dev": "tsx watch src/index.ts"
```

This creates a shortcut named dev. tsx runs the TypeScript file, watch restarts the server on every save, and src/index.ts is the file to run. Then run:

```
npm run dev
```

You should see: Server running on http://localhost:3001. Note that localhost:3001 alone shows "Cannot GET /" because no route exists there. Open http://localhost:3001/users instead.

API Endpoints

Method	Endpoint	Description	Success status
GET	/users	List all users	200

Method	Endpoint	Description	Success status
GET	/users/:id	Get one user	200 (404 if missing)
POST	/users	Create a user (body: name, email)	201 (409 if email exists)
PUT	/users/:id	Update a user (body: name, email)	200
DELETE	/users/:id	Delete a user	204

Step 12: Test With Postman

- GET http://localhost:3001/users returns an empty array.
- POST the same address: Body tab, choose raw, change Text to JSON, and send:

```
{
  "name": "King",
  "email": "king@example.com"
}
```

- It returns 201 with id 1 and a createdAt time.
- GET /users to see the list, then GET /users/1.
- PUT /users/1 with a changed name.
- DELETE /users/1 returns 204.
- POST the same email twice to see the 409 error.

Step 13: Connect the Frontend

1. Keep the backend running with npm run dev.
2. In the frontend folder, check that .env.local contains NEXT_PUBLIC_API_URL=http://localhost:3001.
3. Run npm run dev in the frontend folder and open http://localhost:3000.
4. Add, edit, and delete a user, and try a duplicate email to see the red error message.

The frontend needs no code changes, because it only talks to the Express API.

Step 14: Run Everything With Docker Compose

Docker Compose starts the database and the backend together with one command: `docker compose up --build`. You no longer type docker run or npm run dev yourself.

All three files go in the backend_postgresql folder, next to package.json.

14.1 Add a production start script

In package.json, inside "scripts", add this next to dev:

```
"start": "tsx src/index.ts"
```

It is the same as dev but without watch, so it does not restart on every save. Use dev while coding and start in deployment.

14.2 Dockerfile

Create a file named exactly `Dockerfile`, with no extension. Docker follows its lines in order to build the backend image.

```
FROM node:22
WORKDIR /app
COPY package*.json ./
RUN npm install
COPY . .
RUN npx prisma generate
EXPOSE 3001
CMD ["sh", "-c", "npx prisma migrate deploy && npm start"]
```

- **FROM node:22**: start from an image that already has Node.js.
- **WORKDIR /app**: work inside the /app folder in the container.
- *COPY package.json and RUN npm install**: copy the package files first and install packages.
- **COPY . .**: copy the rest of the project.
- **RUN npx prisma generate**: create the Prisma client inside the image.
- **EXPOSE 3001**: documents that the backend uses port 3001.
- **CMD**: when the container starts, apply saved migrations with migrate deploy, then run npm start.

14.3 docker-compose.yml

Create a file named `docker-compose.yml`:

```
services:
  db:
    image: postgres:16
    environment:
      POSTGRES_USER: postgres
      POSTGRES_PASSWORD: postgres
      POSTGRES_DB: usersdb
    ports:
      - "5432:5432"
    volumes:
      - pgdata:/var/lib/postgresql/data
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U postgres -d usersdb"]
      interval: 3s
      timeout: 3s
      retries: 10

  backend:
    build: .
    ports:
      - "3001:3001"
    environment:
      DATABASE_URL: postgresql://postgres:postgres@db:5432/usersdb
      PORT: 3001
    depends_on:
      db:
        condition: service_healthy

volumes:
  pgdata:
```

- **services**: the two containers, db (PostgreSQL) and backend (your Express app).
- **Host name**: inside Compose, each service name works as an address. The backend reaches PostgreSQL at **db**, not localhost, because localhost inside the backend container means the backend itself.
- **Volume pgdata**: a storage area Docker keeps outside the container. It is connected to PostgreSQL's data folder, so your users survive when containers are restarted or removed.

- **Health check**: makes the backend wait until PostgreSQL reports it is ready.
- **depends_on**: starts the backend only after the db health check passes.

14.4 Create .dockerignore

Create a file named `.dockerignore` containing this line, so Docker does not copy your local modules into the image:

```
node_modules
```

14.5 Stop the old container, then run Compose

The earlier container named usersdb uses port 5432, so remove it first. Also stop npm run dev if it is running, because it uses port 3001.

```
docker stop usersdb
docker rm usersdb
docker compose up --build
```

The old container had no volume, so its test data is deleted with it. The Compose database starts empty. When you see "Server running on http://localhost:3001", open http://localhost:3001/users, which should return an empty array.

14.6 Daily Compose commands

Command	What it does
docker compose up --build	Build the backend and start everything
docker compose up -d	Start in the background
docker compose down	Stop and remove containers, data is kept in the volume
docker compose down -v	Also deletes the volume, which wipes the database

In deployment you do not run npm run dev. You run docker compose up, and the Dockerfile starts the backend for you.

Troubleshooting

- **Cannot connect to database**: check that Docker Desktop is running and that docker ps shows usersdb. Check that DATABASE_URL matches your docker run values and port.
- **Port 5432 already in use**: use a different left port, for example -p 5433:5432, and update DATABASE_URL to port 5433.
- **Prisma update available message**: harmless, a newer Prisma version exists.
- **Failed to fetch in the frontend**: check that the backend is running and that .env.local says http://localhost:3001. Restart the frontend after editing .env.local.

Key Concepts Learned

- **ORM**: maps tables to models, rows to objects, columns to properties.
- **Migration**: a saved, versioned step in your database history. Do not edit the migration files.
- **Adapter**: the bridge between Prisma and a specific database.
- **Docker container**: a packaged program, here PostgreSQL, running isolated on your computer.
- **Port mapping**: left number is your computer, right number is inside the container.
- **DATABASE_URL**: the database address, set as an environment variable so the same code runs everywhere.

Next Steps

- Deploy the Compose setup to a server or VPS, and change the PostgreSQL password and DATABASE_URL to strong values stored as environment variables.
- Deploy with the database on Neon or Supabase.